

Лабораторная работа №8

Отчет

Устинова Виктория Вадимовна

Содержание

1	Цель работы	5
2	Задание	6
2.1	Выполнение	7
3	Выводы	8

Список иллюстраций

2.1	шифрует две телеграммы P_1 и P_2 одним ключом;использует извест- ный шаблон P_1	7
2.2	Это доказывает, что, зная только шифротексты и формат первого сообщения, можно прочитать часть второго сообщения, не расшиф- ровывая ключ	7

Список таблиц

1 Цель работы

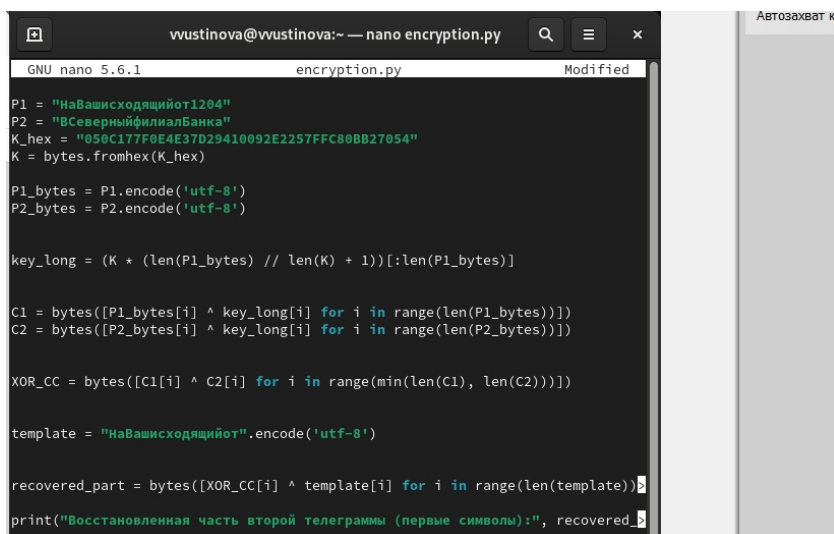
Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом

2 Задание

1. Даны два открытых текста (телеграммы P_1 и P_2) и один секретный ключ K длиной 20 байт. Оба текста зашифрованы одним и тем же ключом в режиме однократного гаммирования (XOR).
2. Требуется, не зная ключа и не пытаясь его подобрать, восстановить содержание обоих текстов, используя только полученные шифротексты C_1 и C_2.
3. Уязвимость: при шифровании двух разных сообщений одинаковым ключом выполняется соотношение (ключ сокращается, так как $K \oplus K = 0$).
4. Атака основана на том, что формат первого сообщения частично известен (фиксированный шаблон «НаВашисходящийот...»). Используя это, из равенства восстанавливаются символы второго сообщения на позициях известного шаблона.
5. Полученный осмысленный фрагмент второго сообщения позволяет достроить его полностью, а затем восстановить и первое сообщение. Таким образом, оба текста читаются без расшифровки ключа.
6. В ходе работы разработано приложение на Python, которое:
 - шифрует оба текста заданным ключом;
 - выполняет описанную атаку без использования ключа;
 - выводит восстановленный фрагмент второго сообщения, что доказывает успешность взлома

2.1 Выполнение

Реализация атаки на языке Python Здесь показан код, который (рис. 2.1).



```
GNU nano 5.6.1 encryption.py Modified
P1 = "НаВашисходящий1204"
P2 = "ВСеверныйфилиалБанка"
K_hex = "050C177F0E4E37D29410092E2257FFC80BB27054"
K = bytes.fromhex(K_hex)

P1_bytes = P1.encode('utf-8')
P2_bytes = P2.encode('utf-8')

key_long = (K * ((len(P1_bytes) // len(K) + 1)))[:len(P1_bytes)]

C1 = bytes([P1_bytes[i] ^ key_long[i] for i in range(len(P1_bytes))])
C2 = bytes([P2_bytes[i] ^ key_long[i] for i in range(len(P2_bytes))])

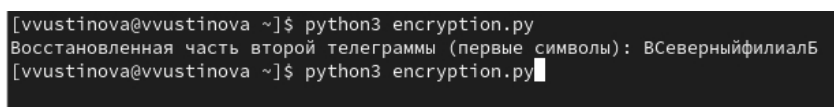
XOR_CC = bytes([C1[i] ^ C2[i] for i in range(min(len(C1), len(C2)))])

template = "НаВашисходящий".encode('utf-8')

recovered_part = bytes([XOR_CC[i] ^ template[i] for i in range(len(template))])
print("Восстановленная часть второй телеграммы (первые символы):", recovered_part)
```

Рис. 2.1: шифрует две телеграммы P_1 и P_2 одним ключом;использует известный шаблон P_1

Результат работы программы В консоли видно восстановленный фрагмент второй телеграммы:«всеверныйфилиалб...»(рис. 2.2).



```
[vvustinova@vvustinova ~]$ python3 encryption.py
Восстановленная часть второй телеграммы (первые символы): ВСеверныйфилиалБ
[vvustinova@vvustinova ~]$ python3 encryption.py
```

Рис. 2.2: Это доказывает, что, зная только шифротексты и формат первого сообщения, можно прочитать часть второго сообщения, не расшифровывая ключ

3 Выводы

У нас получилось освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом